



A multiple ant colony system with random variable neighborhood descent for the dynamic vehicle routing problem with time windows

Orivalde Soares da Silva Júnior¹ · José Eugênio Leal² · Marc Reimann³

© Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract

This paper proposes a framework to solve the dynamic vehicle routing problem with time windows. This problem involves determining the minimum cost routes of a homogeneous fleet for meeting the demand for a set of customers within time windows. In addition, new customers can be assigned to vehicles during the execution of the routes. A framework is based on two phases: a priori where the routes are obtained for the known customers using static routing, and a posteriori where routes are re-optimized repeatedly during the planning horizon either continuously or periodically. The framework was validated using seven algorithm variants based on insertion heuristic, ant colony optimization, variable neighborhood descent, and random variable neighborhood descent, which were adapted to solve a posteriori phase. The best algorithm is a hybrid version that combines an improved version of the multiple ant colony systems with a random variable neighborhood descent. Computational results show that most of the algorithms are competitive regarding the state of the art with the best results in the objective of minimizing the number of unserved customers.

Keywords Dynamic vehicle routing · Time windows · Ant colony optimization · Random variable neighborhood descent

Communicated by V. Loia.

Electronic supplementary material The online version of this article (<https://doi.org/10.1007/s00500-020-05350-4>) contains supplementary material, which is available to authorized users.

✉ Orivalde Soares da Silva Júnior
orivalde@ime.eb.br

José Eugênio Leal
jel@puc-rio.br

Marc Reimann
marc.reimann@uni-graz.at

¹ Department of Fortification and Construction Engineering, Military Institute of Engineering, Praça General Tibúrcio, 80, Urca, Rio de Janeiro, RJ, Brazil

² Department of Industrial Engineering, Pontifical Catholic University of Rio de Janeiro, Rua Marquês de São Vicente, 225, Gávea, Rio de Janeiro, RJ 22451-900, Brazil

³ Department for Production and Operations Management, University of Graz, Universitätsstrasse 15/E3, 8010 Graz, Austria

1 Introduction

Investment in vehicle routing research is relevant from both theoretical and practical viewpoints. Regarding the theoretical viewpoint, the vehicle routing problems and their variants belong to the NP-hard class (Lenstra and Rinnooy Kan, 1981). Thus, the efficient solution of these problems represents a challenge for researchers who need to develop methods to achieve good-quality solutions in the least possible computational time. From a practical viewpoint, with advances in information technology, it has become possible to manage the fleet in real time. In this environment, dispatchers often need to reconfigure the routes of vehicles in real time to improve efficiency and enhance the quality of service when serving new requests. This variant is known as the dynamic vehicle routing problem (DVRP).

A widely used approach to solve the DVRP is to divide it into two phases: a priori and a posteriori. In a priori phase, routes are obtained for the already known customers, and the problem can be solved by the method of static routing. In a posteriori phase, routes are re-optimized repeatedly during the planning horizon either continuously or periodically. In this phase is decomposing the problem

in a sequence of n static and deterministic problems (DP_n) with various solutions (s_n) during the workday (WD). This is an iterative process that evolves concerning time, and new customer entries must be entered in the procedure during its execution. The size n depends on the strategy used in a posteriori phase, which can be *continuous* re-optimization or *periodic* re-optimization. In *continuous* re-optimization, n is equal to the number of new customers. Under this setting, a new static problem is solved whenever a new order becomes known. The *periodic* re-optimization approach uses the concept of splitting the working day into time slices of equal sizes. Therefore, a working day with a total of WD seconds is divided into n_{ts} time slices, each one with f seconds, $f = WD/n_{ts}$. One static problem is solved for each time slice. The advantage of the *periodic* re-optimization is that it does not require the execution of the routing method whenever any data of the problem are updated, reducing the computational effort. The main disadvantage is that the delay in making a decision could result in any of the following: an increase in the total distance of the routes, an increase in the number of vehicles, or an increase in the infeasibility of the solution when service for a customer can no longer be performed within the prescribed time window.

This paper carries out a theoretical study, the development of a new solution approach, and a performance evaluation (presented in Sect. 5) of optimization algorithms (meta-heuristics) aimed at solving the dynamic vehicle routing problem with time windows (DVRPTW). The two main contributions of this paper are the proposal of a framework to solve the DVRPTW and the proposal of seven new algorithms and the good results in terms of minimizing the number of unserved customers and outperforming the results of all approaches in the literature. Those contributions are not trivial and are not found in other works in the literature about the DVRPTW.

The proposed framework is a generic algorithm for DVRPTW that combines the most used strategies and methods to solve this problem. The strategies can be based on continuous re-optimization or periodic re-optimization, and the methods can be the insertion or re-routing. Precisely, we consider both periodic re-optimization and continuous re-optimization. Moreover, for a posteriori phase we consider both insertion and re-routing heuristics, based on heuristics and meta-heuristics. The framework was tested with seven new algorithms based on the insertion heuristic push-forward insertion heuristic—PFIH (Solomon 1987 and Thangiah et al. 1994) and the meta-heuristics variable neighborhood search—VNS (Mladenovic and Hansen 1997), random variable neighborhood descent—RVND (Subramanian 2012), and multiple ant colony system with random variable neighborhood descent—MACS-RVND (Silva Júnior and Leal 2020).

The paper is organized as follows: Sect. 2 presents the literature review about the DVRP and DVRPTW. In Sect. 3, we present the definition of the DVRPTW. In Sect. 4, we present the seven proposed algorithms to solve the DVRPTW. In Sect. 5, the computational results are presented. Finally, in Sect. 6, conclusions and paper contributions to the state of the art are presented.

2 Literature review

The earliest work was published by Psaraftis (1980), with the development of a dynamic programming approach to the problem of collections and deliveries with time windows, better known as the Dial-a-Ride Problem (DARP). The author also presents several differences between static and dynamic problems (Psaraftis 1988 and 1995).

Pillac et al. (2013) conducted an extensive review of dynamic routing problems. These authors also proposed a taxonomy based on the concepts of the evolution of information and information quality defined by Psaraftis (1995). *Information evolution*: In the static problem, the information does not suffer modification nor is updated. In the dynamic problem, new information is revealed during the execution of routes, for example, with the arrival of customer requests. *Quality of information*: This parameter reflects the possible uncertainty in the available data, which may be known with certainty (deterministic), be known with uncertainty (forecast), follow a probabilistic distribution (stochastic), or be totally unknown. Depending on the problem and the available technology, the routes of the vehicles can be either statically (a priori) or dynamically (a posteriori) assigned. From a static perspective, the problem is to determine a priori a set of robust routes that will undergo minor changes during their execution. From the dynamic perspective, the problem consists of assigning routes to vehicles in real time, communicating new destinations, while the vehicles are on route to serve other customers who are already assigned. Using this taxonomy, one can distinguish between the following four problem classes: *Static and deterministic problems*: All of the inputs are known in advance, and the routes of the vehicles do not change once they are running. *Static and stochastic problems*: These problems are characterized by partially known inputs as random variables, which usually follow a Poisson distribution, e.g., stochastic demand. *Dynamic and deterministic problems*: These problems are also referred to as online or real-time routing (Ichoua et al. 2000). The support of real-time communication technologies is essential for communication between vehicles and the decision maker. Some or all of the inputs are unknown and dynamically revealed overtime during the execution of the routes (Psaraftis 1995). *Dynamic and stochastic problems*:

All or some of the inputs are unknown and dynamically revealed during the execution of the routes, but in this case, the stochastic knowledge about the revealed input is considered (Ritzinger et al. 2016). This information helps to make predictions about the next request occurrences. The DVRP that is discussed in this paper is classified as a dynamic and deterministic problem. A widely used approach to solve this problem is to divide it into two phases: a priori and a posteriori. In a priori phase, routes are obtained for the already known customers, and the problem can be solved by the method of static routing. In a posteriori phase, routes are re-optimized repeatedly during the planning horizon either continuously or periodically.

In the DVRP with *continuous* re-optimization, route optimization is performed continuously throughout the day, i.e., whenever changes are made to the available data, the routing method immediately produces a new current solution. Yang et al. (2004) addressed the pickup and delivery routing problem in which shipping orders between pairs of points arrive dynamically. The authors propose an algorithm called MyOpt, which is a moving horizon approach based on linear programming that is applied whenever a new customer is known.

Lackner (2004) developed four meta-heuristic methods for the dynamic vehicle routing problem with time windows (DVRPTW). The first and second, named ES1-DVRPTW and ES2-DVRPTW, respectively, are based on genetic algorithms. The third, called MACS-DVRPTW, is based on ant colony optimization inspired by the MACS-VRPTW algorithm (Gambardella et al. 1999). The fourth, called AS-DVRPTW, is based on simulated annealing. To compare the results, the author created extensions to the 56 instances of Solomon (1987) for five different degrees of dynamism, totaling 280 instances. The author concluded that the method presenting the best results was the ES1-DVRPTW, followed by MACS-DVRPTW, ES2-DVRPTW, and AS-DVRPTW.

Hong (2012) developed a method based on the large neighborhood search (LNS) introduced by Shaw (1998) and applied it to the DVRPTW. This method works by destroying (removing nodes from) and repairing (inserting nodes into) the current solution using different operators to destroy and repair. The neighborhood is called large because a node that is removed may be reinstated in the same source route or any other route of the current solution. In addition to this process, local searches are applied to various neighborhoods. The author compared the results with Lackner (2004) and Yang et al. (2004) results and concluded that the method gives better results with less computational effort.

Pillac et al. (2012) developed pALNS, a parallel version of the adaptive large neighborhood search (ALNS) proposed by Pisinger and Ropke (2007). The ALNS adds an

adaptive layer that randomly selects destruction and removal operators depending on their past performance, automatically adjusting the algorithm to the instance data. The authors compared the results with the work of Lackner (2004) and Hong (2012) and concluded that the pALNS can reach the state of the art and brings improvements up to 12% compared to previous results.

Barkaoui and Gendreau (2013) develop an adaptive evolutionary approach that uses a genetic algorithm in an adaptive process called adaptive hybrid genetic algorithms (AHGA). The algorithm automatically and dynamically performs auto-configuration of the GA operators that are considered to have the highest impact on solution quality: crossover, mutation, and selection. The objectives considered for DVRPTW were the minimization of customer service denial (or maximization of service level), total lateness, and the total traveled distance. They compare the adaptive version of a hybrid genetic algorithm with the non-adaptive one. The authors indicate that there is currently no reference benchmark for DVRPTW at the moment and they adapt the Solomon (1987) benchmark for VRPTW generating randomly request occurrence times as was done by Gendreau et al. (1999). The results show the ability of the operator combination adaptation approach for producing solutions that are superior to hand-tuning and other adaptive methods.

Schyns (2015) developed two algorithms based on the MACS-VRPTW algorithm (Gambardella et al. 1999) and the ACS-DVRP algorithm (Montemanni et al. 2005) to solve the dynamic vehicle routing problem with time windows and split delivery and heterogeneous fleet (DVRPTWSD). The author applied the algorithms to optimize the journey of refueling trucks. The objective function was the responsiveness, which was defined as completing a delivery as soon as possible within the time window, such that the customer or the vehicle may restart its activities. The difference between the two proposed algorithms was the re-optimization strategy, in that the first is continuous and the second is periodic. The initial solution was changed from the classical approach of the nearest neighbor heuristic to the first in first out (FIFO) and another variant in which the preferred vehicle is the one which incurs the smallest responsiveness for the customer (FIFO2), and the local search was performed with the CROSS exchanges (Taillard et al. 1997). Pheromone conservation was applied as in Montemanni et al. (2005). The results were compared to the Kilby et al. (1998) benchmark used by Montemanni et al. (2005) for DVRP and the Solomon (1987) instances for DVRPTW.

Ozbaygin and Savelsbergh (2019) propose an iterative framework using the branch-and-price algorithm developed for the dynamic vehicle routing problem with roaming delivery locations (DVRPRDL) to re-optimize the vehicle

routes every time the itinerary of a customer is updated. The a priori phase is solved using a branch-and-price solutions, and a posteriori phase is solved by re-using, suitably modified, columns generated during preceding branch-and-price solutions. The computational experiments were realized with 80 instances based on VRPRDL (Ozbaygin et al. 2017) with up to 60 customers and demonstrate the viability of using branch and price for solving.

In the DVRP with *periodic* re-optimization, the optimization is performed throughout the day at regular intervals. The time between the applications of the routing method can be represented by a fixed time interval, also known as decision epochs (Chen and Xu 2006) or time slices (Kilby et al. 1998 and Montemanni et al. 2004). Chen and Xu (2006) developed a dynamic column generation algorithm (DYCOL) for a set partitioning model of the DVRPTW. The authors introduced the concept of decision times over the planning horizon, which are the times during which the optimization process is executed. The novelty of this approach is using columns that are generated at the previous decision time, which greatly reduces the computational time.

Montemanni et al. (2005) developed an ant colony system (ACS) to solve the DVRP. Similar to Kilby et al. (1998), the approach of Montemanni et al. (2005) uses time slices, i.e., the day is divided into periods of equal duration. New requests are accumulated for each time slice, and when it finishes, route optimization is performed using a static routing method. An interesting feature of this approach is the use of a strategy that accomplishes the transmission of pheromones that are obtained in the optimization of a time slice to the next time slice. Thus, the array of pheromones is not reset every time, but rather it is reused for the next solutions. Over time, the older pheromones have less importance to the ants in the implementation of new solutions for the exploration of new paths. A similar approach was used by Rizzoli et al. (2007), Gambardella et al. (2003), Silva Júnior and Leal (2009, 2013), Veen et al. (2013), and Yang et al. (2017).

In addition to the strategies of *continuous* and *periodic* re-optimization, an important choice is the type of solution method of partial routing problems, which is used to tackle a posteriori phase. These methods can be classified into insertion and re-routing methods. Insertion methods construct the solution through consecutive insertions of dynamic customers in the current solution (found in a priori step) as they are requested. Insertion attempts are made between all possible pairs of nodes, and the chosen insertion will be one that does not violate the constraints of the problem and has the lowest cost. An additional constraint

in the dynamic case of the problem is the prohibition of insertions between customers that have already been committed or serviced. A possible and well-known problem with these methods is that—depending on the degree of dynamism—a lot of dynamic customer requests may have to be rejected to preserve feasibility. Re-routing methods build, destroy, and rebuild the solution several times. In a priori phase, a solution is constructed to meet all of the customers. Over time, customers are served or committed by the vehicles, and new dynamic customers occur. In a posteriori phase, all of the customers that have not been served or committed are removed from the solution (destruction), and the solution is rebuilt. In the reconstruction step, each route must start from the depot or the last served or committed customer. If the routing method that is used has a memory of the solutions that have already been created, this information can be used to decrease the computational time (Silva Júnior and Leal 2013). One such application was proposed by Taillard et al. (2001), using an adaptive memory with good solutions. This memory helps to reduce the computational time that is required to find a new solution. However, the implementation of this method becomes more complex. This approach has been used by Gendreau et al. (1999) to adapt the parallel version of the tabu search (Taillard et al. 1997) method to solve the VRPTW applied to express postal services. This approach maintains a set of good routes in an adaptive memory that is used to generate initial solutions for the tabu search method. Whenever a new customer is known, possible inclusion in all of the solutions of the adaptive memory is checked to decide whether to accept or reject the new customer. Using these re-routing methods, it is more likely to find feasible solutions with fewer or even zero rejected customer requests.

3 Problem definition

The DVRPTW can be described by a graph $G(N,A)$, where N is a set including all of the nodes, and A is a set containing all of the arcs. The nodes correspond to the customers and the depot, from where the routes depart, and that is usually represented by a node with index 0. Arcs, denoted by (i,j) , represent the paths connecting the nodes i and j . Each arc (i,j) has an associated length denoted here by the time t_{ij} that is required to traverse each arc. The graph is considered to be complete, i.e., there is an arc (i,j) between each pair of nodes i and j for all $j \neq i$; symmetric, i.e., $t_{ij} = t_{ji}$ for all $j \neq i$; and satisfies the triangle inequality.

In the depot, V homogeneous vehicles are available, all with the same capacity K . With each customer i , a demand $d_i \geq 0$ and a service time s_i are associated. Furthermore, each customer also has their time window; this window is given by $[a_i, b_i]$, where a_i represents the earliest moment and b_i the latest instant to start the service. The time when the request of customer i becomes known or the time of the request is denoted by $ta_i \geq 0$. The depot has a demand and service, the time equal to zero, a time window a_0 equal to zero, and b_0 equal to a value that represents the closing time of the depot. The size of the depot time window may also be called workday WD .

The variables are as follows: $x_{ij} \in \{0,1\}$, where a value of 1 represents the use of arc (i,j) and 0 otherwise; $T_i \geq 0$, which represents the arrival time of a vehicle to customer i ; and $e_i \in \{planned, assigned, in_transit, delivered\}$, which is used to control the customer state in the problem. The *planned* state indicated that the customer was provisionally assigned to a certain route and vehicle, but this may be confirmed or changed later. In the *assigned* state, the customer is definitively assigned to a route. The *in_transit* state occurs when a vehicle is on route to meet customer i at the next point in the route. If the customer is being served or has been served, the status is set to *delivered*.

In the VRPTW, the objective of the problem is to find routes with a minimum total cost that meet all customers, respecting the capacity constraints of the vehicles and the time windows. Furthermore, we wish to determine the minimum number of vehicles that are required to perform the task. In the literature, several authors, especially those who propose exact methods, prioritize minimizing the total cost (Roberti 2012 and Ropke 2013). However, most authors hierarchize the objective in a way that minimizes the number of vehicles (NV) as more important than the total distance (TD) (Gambardella et al. 1999 and Lackner 2004). This hierarchy suggests a closer approximation to reality because the fixed cost of allocating a vehicle is generally much larger than the operating cost of the route. In the DVRPTW, some new conditions may be considered in the problem. New vehicles may be used if they are available at the depot. The restriction requiring that all customers are visited may be relaxed, and feasible solutions may be found where some customers are not served. The problem now has a third objective, which is to minimize the number of unserved customers. Then, the objectives may be presented hierarchically as follows: minimize the number of unserved customers (NA), minimize the number of vehicles (NV), and minimize the total distance (TD).

In the context of the Pillac et al. (2013) taxonomy, the DVRPTW discussed in this paper addresses problem that is dynamic and deterministic. A common strategy for dealing with DVRPTW is the so-called least-commitment strategy. Here, two cases concerning the interaction with the planned solution need to be distinguished. The problem is called preemptive if—upon the arrival of a new customer request—the vehicle may be deviated from its current destination to serve the new customer. When the problem is treated as non-preemptive, used in this paper, it means that once the vehicle is on route to its next destination, it must necessarily meet this next point, i.e., deviations or interruptions are not permitted at this stage of the route (Gendreau et al. 1999).

An important aspect of dynamic routing problems is to define a metric for measuring the problem dynamism. This metric is often called the *degree of dynamism* of a problem and is a value within the range $[0,1]$, with higher values indicating more dynamic problems. The first metric to measure the dynamism of a problem was proposed by Lund (1996). The degree of dynamism δ is defined as the ratio between the number of dynamic customers n_d and the total number of requests n_{tot} , as indicated by Eq. (1):

$$\delta = \frac{n_d}{n_{tot}} \quad (1)$$

4 Algorithms for dynamic vehicle routing problem with time windows

In this section, we propose seven algorithmic variants to solve the DVRPTW. We consider both periodic re-optimization and continuous re-optimization. Moreover, for a posteriori phase, we consider both insertion and re-routing heuristics based on the following heuristics and meta-heuristics:

PFIH—Push-forward insertion heuristic proposed by Solomon (1987) and described by Thangiah et al. (1994) and Silva Júnior and Leal (2020).

VNS—Variable neighborhood search created by Mladenovic and Hansen (1997) and using seven inter-route and five intra-route neighborhood structures described by Subramanian (2012). Among the inter-route structures, five are based on the λ -interchanges scheme proposed by Osman (1993) which consists of the exchange of routes between two customers λ , as denoted by *Shift*(1,0), *Shift*(2,0), *Swap*(1,1), *Swap*(2,1), and *Swap*(2,2); one is based on a *CROSS-exchange* operator (Taillard et al. 1997); and one is based on a *K-shift* operator (Subramanian

2012). Among the intra-route structures, three are based on the Or-opt scheme of Or (1976), which repositions δ customers within the same route. Movements with $\delta = 1, 2$, and 3 are used, as denoted by Or-opt1, Or-opt2, and Or-opt3; one is based on the k -opt operator with $k = 2$, denoted by 2-opt (Croes 1958 and Lin 1965); one is based on the *Exchange* operator (Nagy and Salhi 2005).

MACS-RVND—Multiple ant colony system with random variable neighborhood descent proposed by Silva Júnior and Leal (2020), a hybrid meta-heuristic that combines the ACO based on Gambardella et al. (1999) algorithm with local search based on the RVND algorithm (Subramanian 2012), with the same inter-route and intra-route neighborhood structures described in VNS. This algorithm shows very good solutions for 56 benchmark VRPTW instances of Solomon (1987), the efficiency of the algorithm was demonstrated for 29 of these instances, while the best-known solutions were found for 23 instances with the lowest number of vehicles, with a GAP between total distance concerning the best-known solutions of at most 0.62%. In only four instances, solutions with one more vehicle were found but with a total distance less than the best-known solutions, with an average difference of -5.21%.

The approach used to solve the DVRPTW is decomposing the problem in a sequence of n static and deterministic problems DP_n with various s_n solutions during the workday WD . In *continuous* re-optimization strategy, a new DP_n problem is created when new dynamic customers become known, and in the *periodic* re-optimization strategy, a new DP_n problem is created when finishing the current time slice. An important consideration concerns how to deal with dynamically arriving customer requests. All of the new customers that are received before a given time of cutoff tco are accepted and processed during the workday. Customers that are received after the time limit tco are not served in the current day and are assigned to the next working day. For example, if a working day starts at 7 a.m. and ends at 5 p.m. ($WD = 10$ h), the working day is divided into ten time slices ($n_{ts} = 10$) and if the company's policy is to accept new customers at noon, then the $tco = 0.5$ ($10 \cdot 0.5 = 5$ h).

From the literature review carried out in this work, it is observed that all proposed algorithms are based on a single strategy, continuous re-optimization (Yang et al. 2004; Lackner 2004; Hong 2012; Pillac et al. 2012; Barkaoui and

Gendreau 2013; Ozbaygin and Savelsbergh 2019) or periodic re-optimization (Kilby et al. 1998; Montemanni et al. 2005; Chen and Xu 2006; Rizzoli et al. 2007; Gambardella et al. 2003; Silva Júnior and Leal 2009, 2013; Veen et al. 2013; Yang et al. 2016; Schyns 2015). Also, it is observed that routing methods proposed in these works are used in both the a priori and a posteriori phases, thus reducing their flexibility, for example, using a faster method in a posteriori phase, such as the insertion methods. The framework proposed in this work seeks to cover mainly these two gaps in the literature, proposing a generic algorithm that combines the strategies (continuous or periodic) and methods (insertion or re-routing) most used to solve this problem. We also highlight the work of Ozbaygin and Savelsbergh (2019) which was the only one that proposed a framework to solve a variant of the dynamic routing problem (DVRPDL) and which presents some differences concerning the framework proposed in this work: Re-optimization is performed after some dynamic inputs and not after the occurrence of any dynamic input or after the start of a new time slice, and exact and non-heuristic or meta-heuristic algorithms were used. The proposed framework is contextualized in Algorithm 1.

Algorithm 1 is started with a *continuous* or *periodic* re-optimization *strategy* and with an *insertion* or *re-routing method_type*. In the DP_0 problem, the depot and customers that are already known are inserted, and an initial solution s_0 is obtained using some method to solve the VRPTW. An iterative procedure starts in $time = 1$ (discrete variable) and ends in time (WD). New dynamic customers become known and are included in set D . If set D has a customer, then the approach of re-optimization is performed; otherwise, the customer's states are updated. If the approach is *continuous* or *periodic* with the current time corresponding to the beginning of a new time slice, then the counter n is incremented. Then, a new deterministic problem DP_n with the union of the nodes of DP_{n-1} and the nodes of the set D has to be solved. A partial solution s' is created. If the *method_type* is *insertion*, then s_{n-1} is used as a partial solution. Otherwise, a partial solution s_{n-1} is extracted only with committed customers, i.e., those who have the *in-transit*, *delivered*, or *assigned* state. A new intermediate solution s_n is obtained for the static problem DP_n using partial solution s' as an initial solution for new routes. If necessary, new vehicles can be allocated to meet the largest possible number of customers.

Algorithm 1: DVRPTW-Framework

```

1: Procedure DVRPTW-Framework(strategy, method_type)
2:    $time \leftarrow 0$ 
3:    $n \leftarrow 0$ 
4:    $D \leftarrow \{\}$ 
5:    $DP_0 \leftarrow \{\text{depot and known customers with } ta_i \leq time\}$ 
6:   // Get an initial solution using a method of static routing for the VRPTW
7:    $s_n \leftarrow \text{get\_initial\_solution}(DP_0)$ 
8:   For  $time = 1 \dots WD$  do
9:      $D \leftarrow D \cup \{\text{new customers with } ta_i = time\}$ 
10:    If  $D \neq \{\}$  then
11:      If  $strategy = \text{continuous}$  OR ( $strategy = \text{periodic}$  AND  $time \bmod f = 0$ )
12:         $n \leftarrow n + 1$ 
13:         $DP_n \leftarrow DP_{n-1} \cup D$ 
14:         $D \leftarrow \{\}$ 
15:        If  $method\_type = \text{insertion}$  then
16:           $s' \leftarrow s_{n-1}$ 
17:        Else If  $method\_type = \text{re-routing}$  then
18:          For Each  $i = 1 \dots |s_{n-1}|$  do
19:            // Extract partial solution with the depot and customers  $i$  with status  $e_i \neq \text{planned}$ 
20:            If  $s_{n-1(i)} = 0$  OR  $e_i \neq \text{planned}$  then
21:               $s' \leftarrow s' \cup s_{n-1(i)}$ 
22:            End If
23:          End For each
24:        End If
25:        // Get an intermediate solution using an adapted static routing method
26:         $s_n \leftarrow \text{get\_intermediate\_solution}(s', DP_n, method\_type)$ 
27:      End If
28:    End If
29:    // Update the customers' status
30:    For  $r = 1 \dots V$  do
31:      // For all customers  $i$  of route  $r$ ,  $|r|$  is the number of customers on route  $r$ 
32:      For  $i = 1 \dots |r|$  do
33:         $start\_travel_i \leftarrow T_i - t_{i-1,i}$ 
34:        If  $time \geq start\_travel_i$  then
35:          If  $time < T_i$  then
36:             $e_i \leftarrow \text{in\_transit}$ 
37:          Else
38:             $e_i \leftarrow \text{delivered}$ 
39:          End If
40:        Else
41:          If  $strategy = \text{periodic}$  AND  $start\_travel_i \leq n \cdot f$  then
42:             $e_i \leftarrow \text{assigned}$ 
43:          Else
44:             $e_i \leftarrow \text{planned}$ 
45:          End If
46:        End If
47:      End For
48:    End For
49:  Return  $s_n$ 

```

The states of customers are updated in all of the routes of solution s_n . Specifically, for each route, the state of the current customer in the planned sequence is updated to *in_transit* if the vehicle has already started but not completed its journey to the customer. Alternatively, the state is updated to *delivered*, if the vehicle has started service at the customer. The algorithms could work with only two states:

committed and planned. Yet, to provide more precise information about the customer service situation these other committed states were defined. Under *periodic re-optimization*, we need to take into account the length of the next time slice starting at the *time*. All customers, who are scheduled to be visited (i.e., the journey toward the customer has started) before the end of the current time slice,

are labeled with the status *assigned*. Otherwise, the status of the current customer remains *planned*.

Applying the above algorithms for the static VRPTW to provide solutions for intermediate DVRPTW problems $DP_1, \dots, DP_{n-1}$ requires adaptations that depend on the method used in the a posteriori phase: re-routing or insertion. For the insertion algorithm, inserting a customer between customers i and $i + 1$ is only possible if customer $i + 1$ is not committed, $e_{i+1} \notin \{in_transit, delivered, assigned\}$. Under the re-routing approach, a variant of the MACS-RVND algorithm (Silva Júnior and Leal 2020) is used for re-routing not-committed customers of partial solution s' in a posteriori phase. This modified algorithm is called MACS-RVND-A (Algorithm 2).

controlled. The ACS-CLI needs to allow the inclusion of a new vehicle because in every new subproblem new customers can be included dynamically which cannot always be met with the same number of vehicles of a past solution. The following hierarchy of objectives was applied: minimize the number of unserved customers (ACS-CLI), minimize the number of vehicles (ACS-VEI), and minimize the total distance (ACS-TIME).

The ACS-CLI algorithm is similar to the ACS-VEI algorithm; however, it additionally has its IN_j variables, called IN_CLI_j , which are used to count the number of times the customer j was not included in the solutions obtained by the ACS-CLI. Unlike the ACS-VEI, in ACS-CLI, solutions are constructed with more vehicles than in

Algorithm 2: MACS-RVND-A

```

1. Procedure MACS-RVND-A( $s'$ ,  $DP_n$ ,  $method\_type$ )
2. // #vehicle_number( $s^*$ ) returns the number of vehicles that were used in solution  $s^*$ 
3.  $s^* \leftarrow$  Find the initial solution using PFIH and NN using the partial solution  $s'$  and with  $DP_n$  nodes
4. //  $nimax$  is the maximum iteration number for ACS-VEI and ACS-TIME with no feasible solution
5. //  $ni$  is the current iteration
6.  $ni = 0$ 
7. While the stop condition is not fulfilled do
8.    $v \leftarrow$  number_of_vehicles( $s^*$ )
9.    $s^* \leftarrow$  Execute ACS-VEI( $s^*$ ,  $v-1$ )
10.   $s^* \leftarrow$  Execute ACS-TIME( $s^*$ ,  $v$ )
11.  // After  $nimax$  iterations, allows solutions with an additional vehicle
12.  If  $ni \geq nimax$  then
13.     $s^* \leftarrow$  Execute ACS-CLI( $s^*$ ,  $v+1$ )
14.     $ni = 0$ 
15.  Else
16.     $ni = ni + 1$ 
17.  End If
18. End While

```

The main extensions over the MACS-RVND algorithm are the use of the third colony of ants (ACS-CLI) to minimize the number of unserved customers and the control when this third colony will be used. The ACS-CLI colony is an instance of ACS-VEI (Silva Júnior and Leal 2020), and the name ACS-CLI was used only to define the objective of this colony. In the MACS-RVND-A, the ACS-VEI and ACS-TIME were executed sharing the best solution s^* and the number of iterations ni is incremented. After $nimax$ iterations, the ACS-CLI is executed with one vehicle more than the current vehicles in the best solution to try to serve more customers. The goal of this mechanism is to first reduce the number of vehicles and the total distance with ACS-VEI and ACS-TIME. When the algorithm already has a good-quality solution, it then seeks to increase the number of customers served with ACS-CLI. It is very costly for the ACS-VEI to reduce a vehicle and very easy for the ACS-CLI to increase the number of customers with an additional vehicle, and this balance has to be

the best current solution, which enables the inclusion of new customers in the current solution.

Another difference is the adaptation of the constructive algorithm *AntSimulate* (Silva Júnior and Leal 2020) to solve the problems in the re-optimization phase. The first adaptation is the pre-initialization of each ant tour and memory visiting the depot and the committed customers. The second adaptation is pheromone reuse. As previously noted, the ant colony algorithm uses the concentration of pheromones that are deposited by ants to indicate good solutions to the problem. In the case of MACS-RVND, the algorithm utilizes two matrices that store the resulting pheromones of ACS-TIME and ACS-VEI algorithms. In the MACS-RVND-A algorithm, these matrices are reused to solve the problems in a posteriori phase. This approach is called the pheromone conservation strategy. Once the deterministic problems of the same instance are potentially very similar, this operation prevents the optimization from restarting every time from scratch and contributes greatly

to ensuring the performance of the algorithm that is needed for the dynamic problem.

However, when new dynamic customers are inserted, a partial reduction of the current concentration of pheromones in the matrix is necessary to make arcs toward the new customers more attractive. This pheromone information transfer strategy was inspired by the work of Guntsch and Middendorf (2001) and Oliveira (2009), in which γ_r defines a new parameter that regulates the storage of pheromones. For every pair of customers that appear in both the old and new problems, the pheromone concentration of the corresponding arc is initialized by Eq. (2):

$$\tau_{ij} = (1 - \gamma_r)\tau_{ij}^{\text{old}} + \gamma_r\tau_0 \quad (2)$$

where τ_{ij}^{old} is the value of τ_{ij} referring to the old static problem. Thus, the pheromone matrix is not reset every time but is reused for the next solutions. For each new customer, an entry corresponding to the concentration of initial pheromone τ_0 is created in the pheromone matrix.

Based on the algorithmic components discussed above, we propose several different algorithms combining different re-optimization strategies and methods. The different combinations are shown in Table 1 and show a gradual increase in the degree of sophistication and development effort. Analyzing the relative performance of those algorithms will help us understand: (i) the relative importance of the initial solution for solving dynamic problems; (ii) the value of re-routing over-insertion-type methods for re-optimization; and (iii) the value of continuous re-optimization as compared to periodic re-optimization.

The first four algorithms are based on the insertion method for re-optimization. In variants 2–4, the insertion heuristic is enhanced by a VND improvement phase. Moving from the first to the fourth algorithm variant, the initial solutions get more and more sophisticated, ultimately generating a priori solutions with the MACS-RVND approach.

The last two algorithm variants differ from the first four variants in that their re-optimization is based on re-routing. Consequently, these two variants use the MACS-RVND-A

algorithm for computing intermediate solutions. Finally, these two variants differ from each other concerning the re-optimization strategy.

5 Computational results

The purpose of this section is to show the performance of the algorithms that are proposed for DVRPTW. The algorithms are executed on a PowerEdge T110 II server with a 3.4 GHz Intel Xeon Quad-Core and an 8-GB RAM processor under the Linux platform, with the CentOS 6.3 64-bit distribution. The algorithms were implemented using the Java programming language.

5.1 Comparing the performance of the seven algorithms for DVRPTW

We first compare all of our seven algorithmic variants on the Lackner (2004) DVRPTW instances, to gain more insight into their relative performance under different degrees of dynamism. These instances are extensions of the 56 benchmark instances with 100 customers of Solomon (1987) for the DVRPTW using degrees of dynamism equal to 0.1, 0.3, 0.5, 0.7, and 0.9. For each instance and each degree of dynamism, the author created a new parameter that indicates the time when each customer i is known (ta_i), totaling 280 instances. These instances are classified according to customer location and size of the time windows. In the C1 and C2 classes the locations are clustered, in R1 and R2 classes, they are randomly dispersed, and in RC1 and RC2, they are a mix of both. In the C1, R1, and RC1 classes, the time windows are relatively narrow, and in the C2, R2, and RC2 classes, they are relatively wide.

The MACS-RVND method uses a set of parameters which influence considerably the results of the algorithm. Instead of simply using the parameter values published in the literature, we did some experiments with those parameters. To tune the MACS-RVND, parameters ρ , β , q_0 , and k were varied to check the sensitivity to the

Table 1 Algorithms for DVRPTW

Algorithm name	Re-optimization strategy	Re-optimization method	Initial solution	Intermediate solution
PFIH_PFIH	Continuous	Insertion	PFIH	PFIH
PFIH_PFIH-VND	Continuous	Insertion	PFIH	PFIH + VND
PFIH-VND_PFIH-VND	Continuous	Insertion	PFIH + VND	PFIH + VND
MACS-RVND_PFIH-VND	Continuous	Insertion	MACS-RVND	PFIH + VND
MACS-RVND_Re-routingUC	Continuous	Re-routing	MACS-RVND	MACS-RVND-A
MACS-RVND_Re-routingDT	Continuous	Re-routing	MACS-RVND	MACS-RVND-A
MACS-RVND_Periodic	Periodic	Re-routing	MACS-RVND	MACS-RVND-A

parameter values. The experiments were performed with one instance of each class varying the parameter to be tested and keeping all other parameters with fixed values. For each parameter, a range of values was defined and an increment of 1 or 0.1. The results suggest that the optimal value of q_0 must be greater than 0.5 to maintain the efficiency of the algorithm. This experiment suggests that if there are too many exploring ants (low q_0), they fail to utilize the accumulated knowledge, thereby reducing the search efficiency. For the parameter ρ , the results suggest values between 0 and 0.2, for the parameter β values between 0.7 and 1.0, and for the parameter k values between 8 and 12. This analysis shows that the parameter values by Gambardella et al. (1999) are inside the resulted ranges and we decide to use the same values: $k = 10$, $\rho = 0.1$, $\beta = 1$, and $q_0 = 0.9$. The parameters γ_r that regulate pheromone conservation were tested with values 0.2, 0.3, and 0.4, and the best value was the same in Guntch and Middendorf (2001), $\gamma_r = 0.3$. In the periodic re-optimization strategy, we used the number of time slices $n_{ts} = 50$ and the time of cut off was changed to $t_{co} = 1$ (Kilby et al., 1998). In the MACS-RVND-A, the parameter n_{imax} was tested using values between 0 and 1000. To MACS-RVND_Re-routingUC was defined $n_{imax} = 200$ and to MACS-RVND_Re-routingDT the $n_{imax} = 600$.

In the PFIH_PFIH, PFIH_PFIH-VND, and PFIH-VND_PFIH-VND algorithms, the runtime of a priori phase does not exceed 10 s. In a posteriori phase, the runtime does not exceed 2 s. Because these approaches are deterministic approaches that always return the same solution, each was executed only once for each instance. For the MACS-RVND_PFIH-VND, MACS-RVND_Re-routingUC, MACS-RVND_Re-routingDT, and MACS-RVND_Periodic algorithms, the stopping criterion was defined as 1000 iterations without any improvement. These iterations are defined in the main loop of Algorithms 1 and 2. In the experiments, the runtime of a priori phase does not exceed 5 min, and in a posteriori phase, for each subproblem solved the runtime does not exceed 10 s. Because these algorithms are probabilistic algorithms that may yield different solutions, five runs for each instance were performed, and the best solution was used for comparison.

Table 2 presents the average values per degree of dynamism δ of the best solutions obtained by the algorithms for DVRPTW. For each algorithm, we present the average number of vehicles (NV), the average total distance (TD), the average percentage of unserved customers (UC) as well as the total sum of computational time (CPU(s)). The last column (TS) represents the total sum of each line.

As mentioned above, we are interested in understanding the contribution of the different algorithmic components to the overall algorithm performance. From the last column in

Table 2, we first observe that the simplest algorithm using pure insertion mechanisms for generating the initial as well as the intermediate solutions is clearly the fastest algorithm. Yet it performs very poorly, particularly in the number of vehicles and the total distance. Its good performance in terms of the rejected customers is misleading since this is purely driven by a large number of extra vehicles. Looking at the PFIH_PFIH-VND, PFIH-VND_PFIH-VND, and the MACS-RVND_PFIH-VND algorithms, it can be seen that strengthening the method that generates the initial solution improves the performance in terms of the number of vehicles as well as the total distance traveled. Contrary to that, the number of unserved customers is first reduced but then increases again. This highlights the trade-off between minimizing the number of vehicles and minimizing the number of unserved customers.

More strikingly, the results in Table 2 show the importance of using the continuous re-routing algorithm. The MACS-RVND-Re-routingUC algorithm outperforms all the other algorithms in terms of both the number of vehicles and the number of unserved customers. Particularly, the number of unserved customers is reduced to slightly more than 1, where all the other algorithms reject more than six customers.

Finally, we can observe the benefit of continuous re-routing as compared to periodic re-routing. The former outperforms the latter concerning every criterion. The improvement ranges from around 0.3% in the number of vehicles and total distance traveled to almost 80% in terms of the number of unserved customers. This last, most striking result shows the importance of advance in information. The time delay induced by a periodic approach may lead to the impossibility of accepting certain requests, which might have been accepted otherwise. Summarizing, Table 2 clearly highlights the superior performance of our most sophisticated algorithm MACS-RVND-Re-routingUC. Before we turn to a closer analysis of this algorithm, let us briefly look at the effect the degree of dynamism has on solution quality.

Comparing the results in the different columns, we see that an increasing degree of dynamism δ generally increases the number of unserved customers and also increases the total distance traveled. On the other hand, the effect on the number of vehicles is less clear. The number of vehicles may increase or decrease as a function of δ . This once again highlights the delicate relationship between the number of unserved customers and the required fleet size.

Table 2 Comparison of the results of the different algorithms for the DVRPTW

Algorithm name		Degree of dynamism δ					TS
		0.1	0.3	0.5	0.7	0.9	
PFIH_PFIH	NV	61.69	63.40	65.84	68.48	67.47	326.88
	TD	12766.53	14005.27	14328.77	15064.01	14407.38	70571.96
	UC	0.34	0.93	0.95	1.41	1.93	5.56
	CPU(s)	1.40	1.20	1.20	1.30	1.30	6.40
PFIH_PFIH-VND	NV	52.97	51.26	50.82	51.28	50.29	256.62
	TD	6698.77	6650.03	6745.04	6758.76	6801.58	33654.18
	UC	0.34	1.44	1.61	2.02	2.68	8.09
	CPU(s)	12.90	18.00	18.90	17.50	14.90	82.20
PFIH-VND_PFIH-VND	NV	49.09	51.12	51.72	50.85	50.60	253.38
	TD	6519.46	6594.30	6761.32	6840.84	6734.57	33450.49
	UC	0.34	0.64	1.32	1.99	2.11	6.40
	CPU(s)	25.30	24.50	20.80	18.90	14.90	104.40
MACS-RVND_PFIH-VND	NV	49.26	50.31	51.10	50.12	50.39	251.18
	TD	6328.66	6400.80	6529.99	6645.18	6768.38	32673.01
	UC	0.68	1.61	1.72	2.55	2.08	8.64
	CPU(s)	660.50	516.70	291.40	166.20	21.90	1656.70
MACS-RVND_Re-routingUC	NV	48.58	49.47	50.47	50.10	50.43	249.05
	TD	6391.52	6578.27	6827.96	6847.06	7066.69	33711.50
	UC	0.08	0.21	0.08	0.46	0.50	1.33
	CPU(s)	4806.40	7829.90	8374.20	7443.60	6686.80	35140.90
MACS-RVND_Re-routingDT	NV	49.75	50.13	51.07	50.49	51.00	252.44
	TD	6301.56	6416.46	6591.45	6663.66	6789.09	32762.21
	UC	0.43	0.52	1.14	1.63	2.22	5.93
	CPU(s)	4865.51	7892.53	8416.07	7592.47	6700.17	35466.75
MACS-RVND_Periodic	NV	49.63	50.96	53.10	52.28	51.96	257.93
	TD	6532.49	6706.23	7060.15	7143.88	7334.32	34777.07
	UC	0.18	0.86	1.05	2.07	2.09	6.25
	CPU(s)	3245.10	3794.10	3498.20	3248.40	3061.10	16846.90

5.2 Analyzing the robustness of the MACS-RVND_Re-routingUC algorithm

Above, we have seen that using the MACS_RVND algorithm in both phases clearly outperforms all the other algorithm variants. We will now take a closer look at this algorithm to better understand its performance. First, we look at the robustness of the results obtained. Table 3 shows the best, average, and worst results recorded over the five runs performed for each instance, average aggregated over all instances for the degree of dynamism δ . The full, disaggregated results are presented in Tables 5–9 in Appendix.

Table 3 highlights the remarkable robustness of the results generated by the MACS-RVND_Re-routingUC algorithm. For the average instance, the difference between the best and worst results is only slightly larger than half a

vehicle, 0.25 rejected customers, and about 7% in terms of the total distance traveled. Moreover, these results are rather insensitive to the degree of dynamism δ .

5.3 Comparing the MACS-RVND_Re-routingUC algorithm with the state of the art for the DVRPTW

We now turn to the evaluation of our MACS-RVND_Re-routingUC algorithm in terms of its performance relative to the state of the art of published works for the DVRPTW, which so far consists of the works of Lackner (2004), Hong (2012), and Pillac et al. (2012). These authors did not publish the solution that was obtained for each of the 280 instances. Only the average values per instance class and degree of dynamism values were published. We could not include Ichoua et al. (2000) and Schyns (2015) in this

Table 3 Robustness of the results obtained with the MACS-RVND_Re-routingUC algorithm

δ	Best			Average			Worst		
	NV	TD	UC	NV	TD	UC	NV	TD	UC
0.1	8.29	1073.15	0.02	8.50	1091.81	0.04	8.66	1118.96	0.05
0.3	8.45	1104.12	0.04	8.67	1124.06	0.09	8.95	1148.18	0.14
0.5	8.61	1146.39	0.02	8.84	1170.29	0.10	9.13	1201.23	0.21
0.7	8.55	1148.53	0.09	8.84	1190.70	0.21	9.18	1230.23	0.30
0.9	8.61	1187.25	0.09	8.94	1230.62	0.20	9.18	1276.26	0.34

Table 4 Comparison of the results of the MACS-RVND_Re-routingUC with state of the art for DVRPTW

Class	δ	MACS-RVND_Re-routingUC		Pillac et al. (2012)		Hong (2012)		Lackner (2004)	
		TD	UC	TD	UC	TD	UC	TD	UC
C1	0.1	903.73	0.00	850.60	0.11	895.80	0.22	996.40	0.00
	0.3	949.06	0.00	874.90	0.11	962.10	0.33	1066.90	0.00
	0.5	986.60	0.00	903.40	0.11	1001.20	0.22	1236.10	0.00
	0.7	1001.14	0.00	919.10	0.11	1031.70	0.22	1261.30	0.00
	0.9	1052.80	0.00	929.90	0.11	1039.80	0.22	1479.60	0.00
C2	0.1	598.85	0.00	597.20	0.00	594.70	0.00	629.10	0.00
	0.3	607.38	0.00	597.60	0.00	651.40	0.00	632.30	0.04
	0.5	644.24	0.00	604.00	0.00	605.00	0.00	689.30	0.13
	0.7	630.85	0.00	619.20	0.00	636.50	0.00	743.80	0.21
	0.9	657.79	0.00	625.70	0.00	636.80	0.00	792.50	0.29
R1	0.1	1267.44	0.08	1197.40	0.25	1257.10	0.17	1278.10	0.47
	0.3	1304.79	0.08	1212.90	0.80	1286.60	0.58	1337.90	0.72
	0.5	1358.13	0.08	1225.00	1.25	1295.80	0.67	1330.00	0.78
	0.7	1346.81	0.33	1237.30	1.71	1331.30	1.75	1336.10	0.94
	0.9	1427.12	0.25	1230.10	2.55	1335.90	2.33	1278.30	0.75
R2	0.1	996.97	0.00	893.00	0.00	950.00	0.09	1052.90	0.03
	0.3	1012.13	0.00	915.60	0.00	985.50	0.00	1085.40	0.15
	0.5	1051.65	0.00	948.60	0.00	1016.50	0.00	1138.80	0.21
	0.7	1051.02	0.00	967.70	0.00	1032.00	0.09	1116.90	0.30
	0.9	1063.76	0.00	981.70	0.00	1047.80	0.09	1193.30	0.52
RC1	0.1	1465.01	0.00	1389.40	0.04	1436.20	1.13	1426.90	0.46
	0.3	1508.58	0.13	1421.50	0.28	1492.20	1.13	1439.70	0.42
	0.5	1573.11	0.00	1463.40	0.23	1514.70	1.38	1448.10	0.46
	0.7	1599.18	0.13	1470.10	0.58	1511.30	1.88	1488.40	0.58
	0.9	1605.36	0.25	1495.50	0.51	1513.90	2.00	1475.20	0.42
RC2	0.1	1159.53	0.00	1024.40	0.00	1103.30	0.00	1220.90	0.00
	0.3	1196.34	0.00	1053.10	0.00	1166.00	0.25	1244.90	0.04
	0.5	1214.24	0.00	1060.50	0.00	1190.50	0.13	1244.90	0.00
	0.7	1218.06	0.00	1091.40	0.00	1239.50	0.00	1269.30	0.00
	0.9	1259.86	0.00	1130.30	0.00	1257.20	0.13	1346.80	0.13
TS		33711.50	1.33	30930.50	8.75	33018.30	15.01	35280.10	8.05
GAP (%)				8.99	-84.80	2.10	-91.14	-4.45	-83.48

comparison because these authors did not use Lackner's instances and did not publish the arrival of customer requests ta_i . Instead, they used random values to determine the ta_i and did not publish these values explicitly.

Table 4 summarizes the results of the competing algorithms. Specifically, the penultimate row highlights the total sum of the total distance traveled and the number of unserved customers' overall instance classes and degrees of

dynamism. The last row shows the percentage deviation of the results of our MACS-RVND_Re-routingUC algorithm from the competing algorithms. From Table 4, we observe that our algorithm outperforms the competing algorithms for the number of unserved customers by a large margin of more than 83%. Moreover, we obtain the lowest number of unserved customers for each instance class and each degree of dynamism.

When looking at the average total distance traveled, the picture is more mixed. On the one hand, our algorithm also outperforms the algorithm of Lackner (2004), which performs second best to our approach in terms of the number of unserved customers, by almost 4.5% in the average total distance. For the other two algorithms, the superior performance of our algorithm for the number of unserved customers comes at the cost of an increased average total distance. Specifically, the best algorithm by Pillac et al. (2012) features a 9% lower total distance traveled than our algorithm, while Hong (2012) provides a 2.10% lower distance traveled.

Summarizing, our algorithm has superior performance when compared with these state-of-the-art approaches. When the *cost* of rejecting customers is sufficiently high, our algorithm provides the best result to the instance classes C1, R1, and RC1. On the other hand, when rejecting customers is less problematic, the lowest total distance traveled makes the Pillac et al. (2012) algorithm the preferred choice for all instance classes. We regard the superior performance of our algorithm concerning the objective of minimizing the number of unserved customers (UC) as the second main contribution of this paper.

6 Conclusions

The vehicle routing problem plays a very important role in the fleet management business, and for decades, practical applications were directed only to the static problem. Recent advances in information technology have provided the necessary tools for companies to manage their fleets in real time. However, these new technologies have also introduced more complexity in fleet management, revealing the need for the development of decision support for dynamic vehicle routing systems.

The first contribution was a proposal of a generic algorithm to solve the problem of dynamic vehicle routing with time windows. This algorithm has been used as a framework for seven new algorithms for DVRPTW. These algorithms are based on the insertion algorithm PFIH, the method of variable neighborhood descent (VND), and the multiple ant colony system with random variable neighborhood descent (MACS-RVND). These algorithms were applied to 280 benchmark instances. The algorithm with

the best performance was selected and compared with the best results reported in the literature. Because it is a relatively new problem, the literature of dynamic problems is still scarce and because the authors do not provide all of the necessary data to perform a complete comparison, it was not possible to compare the number of vehicles that were used by the authors. However, a comparison of the average total distance and the average of the total percentage of customers who were not served demonstrated that the algorithm provides high-quality solutions, outperforming the results of all approaches in the literature for the number of unserved customers. We consider this to be the second main contribution of this paper.

Acknowledgements This work was supported by CNPq—National Council for Scientific and Technological Development under Grant No. 140015/2011-6, CAPES—Brazilian Federal Agency for Support and Evaluation of Graduate Education under Grant No. 001, and FAPERJ—Foundation for the Research Support of the State of Rio de Janeiro under Grant No. E-26/202.175/2015.

Compliance with ethical standards

Conflict of interest The authors declare that there is no conflict of interest regarding the publication of this paper.

Ethical approval This article does not contain any studies with human participants or animals performed by any of the authors.

Informed consent Informed consent is obtained from all individual participants included in the study.

References

- Barkaoui B, Gendreau M (2013) An adaptive evolutionary approach for real-time vehicle routing and dispatching. *Comput Oper Res* 40:1766–1776
- Chen Z, Xu H (2006) Dynamic column generation for dynamic vehicle routing with time windows. *Transp Sci* 40(1):74–88
- Croes G (1958) A method for solving traveling salesman problems. *Oper Res* 6:791–812
- Gambardella L, Taillard É, Agazzi G (1999) MACS-VRPTW: a multiple ant colony system for vehicle routing problems with time windows. *New Ideas in Optimization*. Londres, McGraw-Hill, pp 63–76
- Gambardella L, Rizzoli A, Oliverio F, Casagrande N, Donati A, Montemanni R, Lucibello E (2003) Ant colony optimization for vehicle routing in advanced logistics systems. In: *International workshop on modelling and applied simulation, MAS 2003*, SI, pp 3–9
- Gendreau M, Guertin F, Potvin J, Taillard E (1999) Parallel tabu search for real-time vehicle routing and dispatching. *Transp Sci* 33(4):381–390
- Guntsch M, Middendorf M (2001) Pheromone modification strategies for ant algorithms applied to dynamic TSP. In: *Proceedings of EcoWorkshops on applications of evolutionary computing*, vol 2037, *Lecture Notes in Computer Science*, pp 213–222

- Hong L (2012) An improved LNS algorithm for real-time vehicle routing problem with time windows. *Comput Oper Res* 39(2):151–163
- Ichoua S, Gendreau M, Potvin J (2000) Diversion issues in real-time vehicle dispatching. *Transp Sci* 34(4):426–438
- Kilby P, Prosser P, Shaw P (1998) Dynamic VRPs: a study of scenarios. Strathclyde, UK
- Lackner A (2004) Dynamische Tourenplanung mit ausgewählten Metaheuristiken, 47. Göttinger Wirtschaftsinformatik, Cuvillier
- Lenstra J, Rinnooy Kan A (1981) Complexity of vehicle routing and scheduling problems. *Networks* 11:221–227
- Lin S (1965) Computer solutions of the traveling salesman problem. *Bell Syst Tech J* 44:2245–2269
- Lund K (1996) Vehicle routing problems with varying degrees of dynamism. Lyngby, Denmark
- Mladenovic N, Hansen P (1997) Variable Neighborhood Search. *Comput Oper Res* 24(11):1097–1100
- Montemanni R, Gambardella LM, Rizzoli AE, Donati AV (2005) Ant colony system for a dynamic vehicle routing problem. *J Comb Optim* 10:327–343
- Nagy G, Salhi S (2005) Heuristic algorithms for single and multiple depot vehicle routing problems with pickups and deliveries. *Eur J Oper Res* 162:126–141
- Oliveira S (2009) A study of pheromone modification strategies for using ACO on the dynamic vehicle routing problem. Doctoral symposium on engineering stochastic local search algorithms, SLS-DS'09, Brussels, Belgium, pp 6–10
- Or I (1976) Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking. Ph.D. Thesis, Department of Industrial Engineering and Management Sciences, Northwestern University, Evanston, IL
- Osman I (1993) Metastrategy simulated annealing and tabu search algorithms for the vehicle routing problem. *Ann Oper Res* 41(1–4):421–451
- Ozbaygin G, Savelsbergh M (2019) An iterative re-optimization framework for the dynamic vehicle routing problem with roaming delivery locations. *Transp Res Part B Methodol* 128:207–235
- Ozbaygin G, Karasan OE, Savelsbergh M, Yaman H (2017) A branch-and-price algorithm for the vehicle routing problem with roaming delivery locations. *Transp Res Part B Methodol* 100:115–137
- Pillac V, Guéret C, Medaglia A (2012) A fast re-optimization approach for dynamic vehicle routing. *École des Mines de Nantes, Nantes*, pp 1–22
- Pillac V, Gendreau M, Guéret C, Medaglia A (2013) A review of dynamic vehicle routing problems. *Eur J Oper Res* 225(1):1–11
- Psaraftis H (1980) A dynamic programming solution to the single-vehicle many-to-many immediate request dial-a-ride problem. *Transp Sci* 14:130–154
- Psaraftis H (1988) Dynamic vehicle routing problems. In: Assad B (ed) *Vehicle routing: methods and studies*. Elsevier, Holland, pp 223–248
- Psaraftis H (1995) Dynamic vehicle routing: Status and prospects. *Ann Oper Res* 61:143–164
- Ritzinger U, Puchinger G, Hartl RF (2016) A survey on dynamic and stochastic vehicle routing problems. *Int J Prod Res* 54:1–19
- Rizzoli A, Montemanni R, Lucibello E, Gambardella L (2007) Ant colony optimization for real-world vehicle routing problems. *Swarm Intell* 1:135–151
- Roberti R (2012) *Exact Algorithms for Different Classes of Vehicle Routing Problems*. Alma Mater Studiorum-University of Bologna, Bologna
- Ropke S (2013) Branching decisions in branch-and-cut-and-price algorithms for vehicle routing problems. DTU Transport, Disponível em: <http://www.gerad.ca/colloques/ColumnGeneration2012/presentations/session7/Ropke.pdf>, Acesso em 10 de julho de 2013
- Schyns M (2015) An ant colony system for responsive dynamic vehicle routing. *Eur J Oper Res* 245(3):704–718
- Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: Maher M (ed) *Principles and practice of constraint programming*, CP98, Lecture Notes in Computer Science. Springer, New York, pp 417–431
- Silva Júnior O, Leal J (2009) Roteirização dinâmica de veículos com janelas de tempo usando de um algoritmo de colônia de formigas. Congresso de Pesquisa e Ensino em Transportes. Anais do XXIII ANPET, Vitória, ES
- Silva Júnior OS, Leal JE (2013) Algorithms for the static and dynamic vehicle routing problem with time windows. Doctoral Thesis, Departamento de Engenharia Industrial, Pontifícia Universidade Católica do Rio de Janeiro, Rio de Janeiro
- Silva Júnior OS, Leal JE (2020, in press) A multiple ant colony system with random variable neighborhood descent for the vehicle routing problem with time windows. *Int J Syst Manag*
- Solomon M (1987) Algorithms for the vehicle routing and scheduling problems with time windows constraints. *Oper Res* 35(2):254–265
- Subramanian A (2012) Heuristic, Exact and Hybrid Approaches for Vehicle Routing Problem. Tese de Doutorado, Universidade Federal Fluminense, Niterói
- Taillard E, Badeau P, Gendreau M, Guertin F, Potvin J (1997) A tabu search heuristic for the vehicle routing problem with soft time windows. *Transp Sci* 31(2):170–186
- Taillard E, Gambardella L, Gendreau M, Potvin J (2001) Adaptive memory programming: a unified view of metaheuristics. *Eur J Oper Res* 135(1):1–16
- Thangiah S, Osman I, Sun T (1994) Hybrid genetic algorithm simulated annealing and Tabu search methods for vehicle routing problem with time windows. Technical Report, 27, Computer Science Department, Slippery Rock University
- Veen VB, Emmerich M, Yang Z, Bäck T, Kok J (2013) Ant colony algorithms for the dynamic vehicle routing problem with time windows. In: *Natural and artificial computation in engineering and medical applications*. Springer Berlin Heidelberg, pp 1–10
- Yang J, Jaillet P, Mahmassani H (2004) Real-time multivehicle truckload pickup and delivery problems. *Transp Sci* 38(2):135–148
- Yang Z, van Osta JP, van Veen B, van Krevelen R, van Klaveren R, Stam A, Kok J, Bäck T, Emmerich M (2017) Dynamic vehicle routing with time windows in theory and practice. *Nat Comput* 16:119–134

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.